

AI Task 7

Fatima Aleem

Perceptron: A perceptron is the basic building block of a neural network. It's a simple algorithm for binary classification. It takes multiple inputs, multiplies them by weights, sums them up, and passes the result through an activation function to produce an output.

Activation Functions: These functions introduce non-linearity into the neural network, allowing it to learn complex patterns. Some common ones are:

- **ReLU (Rectified Linear Unit):** Outputs the input directly if it's positive, otherwise outputs zero. It's computationally efficient and widely used.
- **Sigmoid:** Squashes the input to a range between 0 and 1. Often used in the output layer for binary classification.
- **Softmax:** Converts a vector of numbers into a probability distribution. Used in the output layer for multi-class classification.

Backpropagation: This is the algorithm used to train neural networks. It works by calculating the error of the output and propagating it backward through the network to adjust the weights of each neuron.

Gradient Descent: This is an optimization algorithm used in conjunction with backpropagation. It helps find the optimal set of weights for the neural network by iteratively adjusting them in the direction of the steepest decrease in the error function.

Implementation of a Perceptron:

```
import numpy as np
```

```
class Perceptron:
```

```
    def __init__(self, num_inputs, learning_rate=0.01):
```

```
        self.weights = np.random.rand(num_inputs) * 0.01
```

```
        self.bias = 0.0
```

```
        self.learning_rate = learning_rate
```

```
    def predict(self, inputs):
```

```
        # Weighted sum of inputs
```

```
        weighted_sum = np.dot(inputs, self.weights) + self.bias
```

```
        # Step function as activation
```

```
    return 1 if weighted_sum > 0 else 0
```

```
def train(self, inputs, target_output):  
    prediction = self.predict(inputs)  
    error = target_output - prediction  
    # Update weights and bias  
    self.weights += self.learning_rate * error * inputs  
    self.bias += self.learning_rate * error  
    return error
```

```
# Example usage
```

```
if __name__ == '__main__':
```

```
    # AND gate data
```

```
    training_data = [  
        (np.array([0, 0]), 0),  
        (np.array([0, 1]), 0),  
        (np.array([1, 0]), 0),  
        (np.array([1, 1]), 1)  
    ]
```

```
    perceptron = Perceptron(num_inputs=2)
```

```
    # Train the perceptron
```

```
    epochs = 100
```

```
    for epoch in range(epochs):
```

```
        total_error = 0
```

```
        for inputs, target_output in training_data:
```

```
            error = perceptron.train(inputs, target_output)
```

```

    total_error += abs(error)

# Stop training if error is zero
if total_error == 0:
    print(f"Training finished after {epoch + 1} epochs")
    break

```

Test the perceptron

```

printf("\nTesting the Perceptron:")

printf("0 and 0: {perceptron.predict(np.array([0, 0]))}")
printf("0 and 1: {perceptron.predict(np.array([0, 1]))}")
printf("1 and 0: {perceptron.predict(np.array([1, 0]))}")
printf("1 and 1: {perceptron.predict(np.array([1, 1]))}")

```

Another Example:

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

ReLU activation function

```

def relu(x):
    return np.maximum(0, x)

```

Sigmoid activation function

```

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

```

Softmax activation function (for a single input vector)

```

def softmax(x):
    e_x = np.exp(x - np.max(x)) # Subtract max for numerical stability
    return e_x / e_x.sum()

```

```
# Generate some data to plot
```

```
x = np.linspace(-5, 5, 100)
```

```
# Plot the activation functions
```

```
plt.figure(figsize=(12, 4))
```

```
plt.subplot(1, 3, 1)
```

```
plt.plot(x, relu(x))
```

```
plt.title('ReLU Activation Function')
```

```
plt.grid(True)
```

```
plt.subplot(1, 3, 2)
```

```
plt.plot(x, sigmoid(x))
```

```
plt.title('Sigmoid Activation Function')
```

```
plt.grid(True)
```

```
plt.subplot(1, 3, 3)
```

```
plt.plot(x, softmax(np.vstack([x, x]).T)) # Softmax expects a 2D array for multiple inputs
```

```
plt.title('Softmax Activation Function')
```

```
plt.grid(True)
```

```
plt.tight_layout()
```

```
plt.show()
```

Here's a simplified conceptual example of how gradient descent updates a single weight:

```
# Conceptual example of Gradient Descent update for a single weight
```

```
current_weight = 0.5
```

```
learning_rate = 0.01
```

```
gradient_of_loss = 2.0 # Assume we calculated this during backpropagation
```

```
# Update the weight
```

```
new_weight = current_weight - learning_rate * gradient_of_loss
```

```
printf("Current Weight: {current_weight}")
```

```
printf("Gradient of Loss: {gradient_of_loss}")
```

```
printf("Learning Rate: {learning_rate}")
```

```
printf("New Weight: {new_weight}")
```